

Rapport de Projet : TravelingApp

Bilan final de la fusion et des développements

Reda & Binôme

17 mai 2026

1. Présentation du Dépôt et Lien

Ce dépôt regroupe l'application finale issue de la fusion complète des deux modules principaux développés au cours du semestre :

- **TravelPath (Java)** : Module de planification intelligente de trajets interactifs.
- **TravelShare (Kotlin)** : Module de réseau social dédié au partage d'expériences de voyages.

Lien du dépôt Git : <https://github.com/redah22/TravelingApp>

2. Parties Développées (Acquises et Fonctionnelles)

2.1. Module Social (TravelShare - Partie 1)

- **Persistance via Base de Données SQLite** : Intégration locale robuste (`DatabaseHelper.kt`) sauvegardant l'intégralité des publications, les images (URI physiques locales Android), les commentaires, et l'inscription sécurisée des utilisateurs.
- **Flux d'actualité** : Création dynamique d'objets `PhotoPost` interfacés avec des éléments de cartes (Lieu, Auteur, Description).
- **Interactions communautaires** : Appui sur la logique Many-To-Many en SQLite pour autoriser les favoris (Likes dynamiques), ainsi que la publication et consultation dynamique de commentaires avec affichage des compteurs correspondants.
- **Authentification fluide** : Flux UI harmonisé couvrant le *Register*, le *Login* et un accès en mode *Anonyme*.

2.2. Module Planification (TravelPath - Partie 2)

- **Moteur de recherche temps-réel** : Saisie avec autocomplétion asynchrone pointant vers l'API `photon.komoot.io` (OpenStreetMap) pour le géocodage des destinations.
- **Générateur d'Aventures asynchrone** : Algorithme multi-paramètres filtrant sur le *budget*, la *durée*, et la *densité d'effort*. L'exploration directe de l'API Overpass génère des Points d'Intérêt (POI) qualitatifs autour de la zone et du secteur d'activité visés.
- **Carte Dynamique et Tuiles OSMDroid** : Rendu visuel d'OpenStreetMap (Mapnik) affichant un tracé (*Polyline*) du parcours suggéré, superposé par des bulles (marqueurs configurés et numérotés) détaillant les distances.

- **Métriques additionnelles** : Connexion dynamique à l'API *Open-Meteo* pour l'affichage précis des prévisions actuelles de la coordonnée GPS cible. Algorithme de *Haversine* permettant de formuler le temps réel de marche entre deux monuments consécutifs.
- **Export document** : Architecture de génération de documents PDF natifs reprenant le planning de la journée heure par heure.

2.3. Logique de Fusion et d'Architecture globale

- **Synchronisation de Sessions asymétriques** : Couplage de la base de données *SQLite* à base de requêtes brutes de TravelShare avec la gestion *SharedPreferences* rapide de TravelPath, assurant à l'utilisateur de n'avoir à créer son compte (**RegisterActivity**) ou se connecter qu'une seule et unique fois quel que soit l'écran sur lequel l'app démarre.
- **Intégrations Backstack** : Modification du cycle de vie des activités (Kotlin) et des fragments (Java) pour conserver une cohérence de la mémoire. Nettoyage absolu lors du bouton de retour au point central **MainActivity**.
- **UX/UI harmonisée** : Tous les Layouts Kotlin (Sombre) ont basculé sur un thème *Light Mode DayNight* pour concorder chromatiquement avec l'interface originelle Violet et Blanc (#7C4DFF) du binôme Java, aboutissant à une charte graphique globale propre et continue.

3. Développements de la phase finale

3.1. Corrections fonctionnelles (Back-end)

- **Authentification unifiée** : Le `LoginFragment` (Java) interroge désormais en cascade les deux sources de données : d’abord les `SharedPreferences` (chemin rapide), puis `DatabaseHelper.kt` via un fallback SQLite si le compte n’est connu que de la base de données (cas d’une réinstallation). La méthode `getInstance` a été annotée `@JvmStatic` pour permettre son appel direct depuis Java.
- **Filtre de visibilité du fil public** : Ajout de la méthode `recupererPostsPublics()` dans `DatabaseHelper` qui ne retourne que les publications ayant le scope `PUBLIC`. Le fil d’actualité (`FeedActivity`) et la carte (`MapFeedActivity`) utilisent désormais cette méthode, garantissant que les posts privés et de groupe n’apparaissent pas dans l’espace public.
- **Contexte utilisateur dans la carte** : `MapFeedActivity` transmettait systématiquement `IS_ANONYMOUS = true` aux fiches détaillées. Ce bug empêchait les utilisateurs connectés d’interagir depuis la carte. Le contexte est maintenant passé correctement via les extras de l’Intent.
- **Synchronisation du fil après publication** : Correction d’un bug de synchronisation entre `postsList` et `currentDisplayList` dans `FeedActivity` : les nouvelles publications sont maintenant insérées dans les deux listes, assurant la cohérence de la recherche immédiatement après une publication.

3.2. Améliorations UX/UI et Accessibilité (Front-end)

- **Dates relatives** : Le `PhotoPostAdapter` calcule désormais des dates relatives (“Il y a 2h”, “Hier”, “Il y a 3j”) au lieu de dates absolues, conformément aux standards UX des réseaux sociaux.
- **Description dans les cartes** : Chaque carte de publication dans le fil affiche maintenant les 2 premières lignes de la description (tronquées par ellipsis), donnant plus de contexte sans alourdir l’interface.
- **Accessibilité TalkBack** : Ajout systématique des attributs `contentDescription` sur toutes les `ImageView` interactives (boutons retour, recherche, profil) et sur les éléments de la liste de publications. Les icônes décoratives sont marquées `importantForAccessibility="no"` pour éviter les doublons vocaux. La zone minimale de toucher des boutons a été portée à 48dp (recommandation Material Design).
- **Thème unifié** : Les deux thèmes (`Theme.Traveling` et `Theme.TravelPath`) utilisent maintenant les couleurs de charte définitives `#7C4DFF`/`#6C63FF` au lieu des couleurs Material par défaut. La barre de navigation système est également colorée en cohérence.
- **Placeholder élégant** : Un drawable vectoriel `placeholder_post.xml` remplace le générique `ic_menu_gallery` pour les publications sans photo.
- **Animation d’entrée** : Les deux cartes de l’écran d’accueil (`HomeFragment`) se révèlent avec une animation *fade + slide-up* décalée, donnant une impression de dynamisme à l’ouverture de l’application.

4. Ce qu’il reste à faire / Pistes d’évolutions

- **Cloud Persistant et Synchronisation distante** : Actuellement, l’intégralité du réseau social TravelShare est restreint au périmètre confiné de la machine (SQLite). Pour valider le concept “Communautaire”, il faudrait implémenter une API REST ou brancher un back-end as-a-Service, tel que *Firebase (Firestore & Storage)*

ou un back-end de type *PostgreSQL/Spring Boot*, afin que le flux de publications et de profils se synchronise en direct entre deux téléphones distincts.

- **Optimisation du Routage (Temps-Réel) :**

Le dessin de l'itinéraire sur la carte d'OSMDroid utilise majoritairement un calcul vectoriel pour tracer ses droits. L'utilisation et la substitution par une API dédiée et spécialisée (ex : *GraphHopper Routing API* ou *Google Directions API*) rendrait les horaires, les routes empruntées et les propositions de détours à pied beaucoup plus réalistes en se fiant aux topologies exactes des routes piétonnes.

- **Notifications Push :**

Les interrupteurs de notification dans l'interface sont actuellement visuels uniquement. L'implémentation complète nécessiterait *Firebase Cloud Messaging* ou un **WorkManager** Android pour déclencher des notifications système réelles lors de nouveaux likes, commentaires ou publications dans les groupes suivis.